Tokenization

From text to token IDs

LLM chapter, session 1. Every token split on these slides is measured, not typed.

Before we start: two links

Watch this. Very, very recommended.

Andrej Karpathy,
Let's build the GPT Tokenizer (YouTube, February 2024, 2 h 13 min).

He writes a GPT-4-style tokenizer from scratch in Python and explains the oddities on today's slides. The list of failures right after the strawberry slide is his.

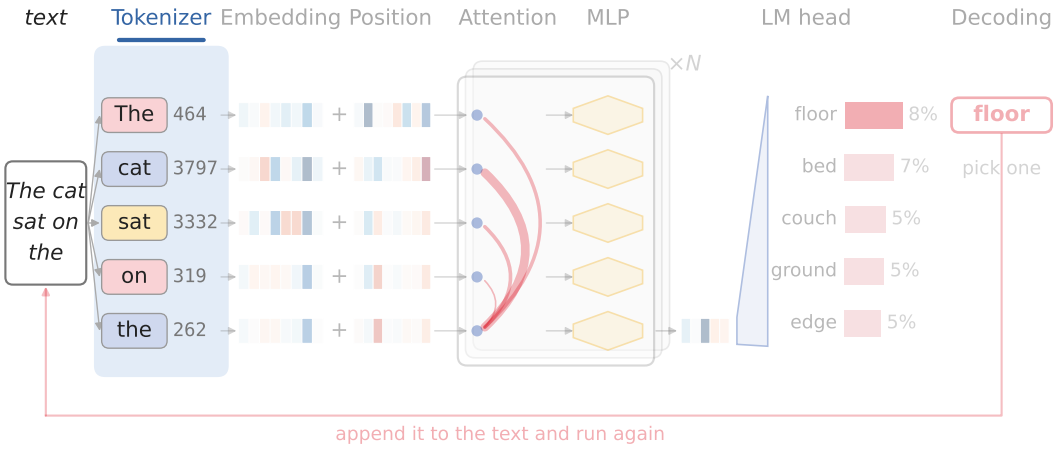
Live demo: tiktokenizer.vercel.app

Paste any text, pick a tokenizer (GPT-2, GPT-4, GPT-4o, Llama 3, Qwen 2.5, ...) and see every token and its ID as you type.

Try:

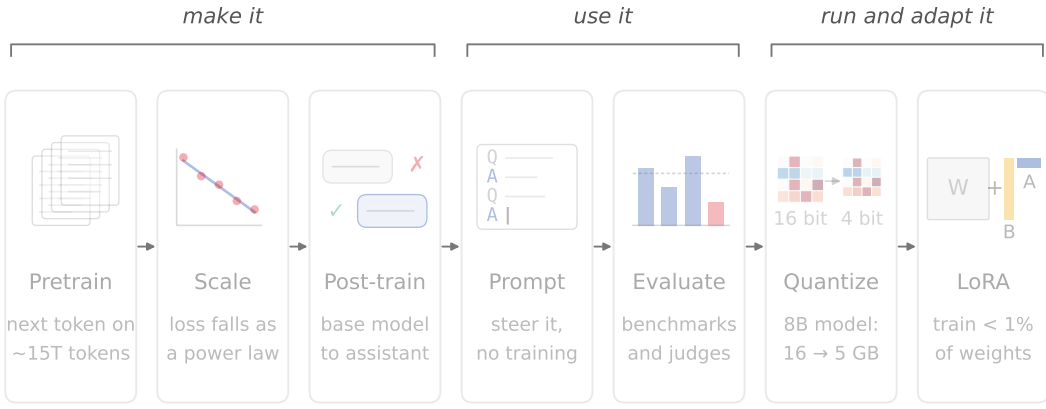
- `How many r's in strawberry?`
- `12345 + 678 = 13023`, then with commas
- an Armenian sentence: GPT-4 vs GPT-4o
- indented Python: GPT-2 vs GPT-4
- `␣SolidGoldMagikarp`: GPT-2 vs GPT-4

Where we are: one forward pass



Today: the first box - how text becomes the integers a model actually reads.

Where we are: the life of a model



Later: where the weights come from, and what you do with them once you have them.

How many r's are in "strawberry"?

Predict: count them.

How many r's are in "strawberry"?

Predict: count them.

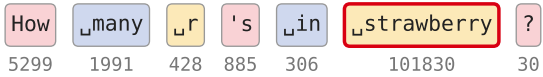
You said 3. In 2024, GPT-4o and Claude both answered **2**, and the failure went viral.
(TechCrunch, 27 Aug 2024)

How many r's are in "strawberry"?

Predict: count them.

You said 3. In 2024, GPT-4o and Claude both answered **2**, and the failure went viral.
(TechCrunch, 27 Aug 2024)

Here is the question as GPT-4o's tokenizer hands it to the model:



You see ten letters. The model sees **one number**: 101830. By the end of today you will know why, and a whole list of other failures with the same cause.

Strawberry is not alone

Karpathy's list from the video. Every row has the same answer.

Why does the LLM ...	**where today**
fail to spell words, or to count the r's in strawberry?	Back to strawberry
fail to reverse a string?	Reverse a word
do worse in non-English languages?	The Armenian tax
fail at simple arithmetic?	next three slides
(GPT-2) have more trouble with Python than it should?	Code whitespace
stop dead when it sees the string <|endoftext|>?	Special tokens
warn me about a "trailing whitespace"?	The trailing-space trap
break when I ask it about "SolidGoldMagikarp"?	Glitch tokens
work better with YAML than with JSON?	Formats cost tokens
What is the real root of suffering?	

Karpathy's answer to every row, the last one included: **tokenization**.

github.com/karpathy/minbpe, `lecture.md`. The fruit even became a code name: OpenAI's first reasoning model, o1, was "Strawberry" inside the company (Reuters, July 2024).

Arithmetic: GPT-2's numbers look random



- Every integer from 0 to 520 is one token. After that it depends on what was frequent: 907 of the numbers 0-9,999 are one token, 88 of them with four digits, 64 of those years.
- This year is two tokens; 2024 and 2025 were one, 2023 was two. 2048 is one: a power of two, common in computing text.
- Neighbours fall apart three different ways: $2249 = 2 + 249$, $2250 = 22 + 50$, $2251 = 225 + 1$. To add them, the model must learn where each digit sits inside each token, token by token.

Millidge (2023), "Integer tokenization is insane". Every split: GPT-2's tokenizer, measured; the years as written mid-sentence, with the space in front.

Predict first: do commas help GPT-4 add?

no commas 

with commas 

GPT-4's tokenizer cuts digits into groups of 3, counted **from the left**. With thousands separators, the groups are counted from the right instead.

Predict: Singh & Strouse (2024) gave GPT-3.5 and GPT-4 additions of 7- to 9-digit numbers, with and without commas. Does the accuracy change, and by how much?

Predict first: do commas help GPT-4 add?

no commas `123` `45` `␣+` `␣` `678` `␣=` `␣` `130` `23`

with commas `12` `,` `345` `␣+` `␣` `678` `␣=` `␣` `13` `,` `023`

GPT-4's tokenizer cuts digits into groups of 3, counted **from the left**. With thousands separators, the groups are counted from the right instead.

Predict: Singh & Strouse (2024) gave GPT-3.5 and GPT-4 additions of 7- to 9-digit numbers, with and without commas. Does the accuracy change, and by how much?

	no commas	with commas
GPT-3.5	75.6%	**97.8%**
GPT-4	84.4%	**98.9%**

By hand: why the commas help

Column addition runs right to left: every column waits for the carry from its right.

No commas. GPT-4 reads `123|45 + 678 = 130|23`.

- `45` holds tens and units, `678` hundreds, tens and units: the chunks do not line up, so no chunk of the answer is "this chunk plus that chunk".
- Where the units digit lands depends on the length: `12345` is `123|45`, `1234` is `123|4`. The model has to work out place value again for every length.

With commas. GPT-4 reads `12,345 + 678 = 13,023`: groups counted from the right, like columns.

- $345 + 678 = 1{,}023$: write 023, carry 1. Then $12 + 1 = 13$.
- Every group of the answer is the matching groups plus a carry: column addition, with 1,000 "digits" instead of 10.

Commas fix the **alignment**, where most errors were. They cannot fix the **order**: the answer is still written left to right (13 before 023), so the carry must be foreseen. That is what still fails.

Outline

Text has to become numbers

From text to bytes

Byte-Pair Encoding

Other subword algorithms

Why LLMs trip over tokens

Whose tokenizer can you see?

Text has to become numbers

A network multiplies numbers. Someone has to decide what the numbers stand for.

Models read IDs, not text

- Every model so far ate numbers: pixels in the CNN chapter, one-hot vectors in [53].
- An LLM reads a list of integers, one per piece of text. The pieces are **tokens**; the list of all possible pieces is the **vocabulary**; a token's **ID** is its row number in that list.

"*The pomegranates arrived fresh and sweet.*"



The open-bracket mark in a chip is a **space**: most tokens carry the space before a word.

[53] did this with a fixed word list ("Tokens, minimally"). Today: which pieces? Next lecture: what the model does with an ID.

Three ways to cut the same sentence

characters
41 tokens



words
7 tokens



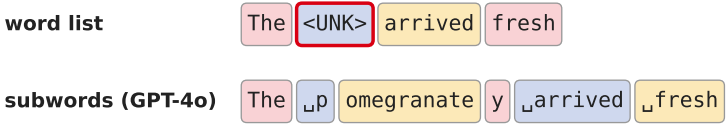
subwords (GPT-4o)
9 tokens



Characters: a tiny vocabulary, very long sequences. **Words:** short sequences, but the vocabulary never ends. **Subwords:** in between. All three are a trade between vocabulary size and sequence length.

Words: the unknown-word problem

- A word list has to stop somewhere. A typo, a name, a new word: anything outside it becomes <UNK>, and the model sees nothing at all.
- Every form is a separate entry: walk, walks, walked, walking. Richly inflected languages need far more entries than English.



A subword tokenizer has no unknown words: the typo just costs one more piece.

Characters: nothing unknown, but long

- About a hundred symbols cover English text, and nothing is ever unknown.
- But the sentences get long: 41 characters vs 9 subword tokens for our sentence. On 5 million characters of English Wikipedia, one GPT-2 token covers **4.6 characters** on average.
- Longer sequences mean more steps for an RNN ([53]) and more compute for every model after it - and one character on its own means almost nothing.

Character-level models work, but they pay about 4.6× the sequence length for that safety.

Subwords: the middle road

- **Frequent** words stay whole: “ arrived”, “ fresh”, “ sweet”.
- **Rare** words split into reusable pieces: “ p” + “omegran” + “ates”. The pieces are not syllables: they are whatever strings were frequent in the tokenizer’s training text.
- No word is ever unknown, and the vocabulary size is a **knob** you set: 50k to 200k pieces in today’s models.

The open question: **which** pieces? Someone has to choose 50,000+ strings out of all possible ones. The answer every GPT uses is an algorithm from 1994.

First, agree on the alphabet

Before cutting text into pieces, decide what text is made of.

Text is a sequence of code points

- Unicode gives every character a number, its **code point**: the letter a is U+0061, the Armenian letter ayb is U+0561.
- Unicode 17.0 (September 2025) defines **159,801** characters: every script, maths symbols, emoji.
- As the starting alphabet for a tokenizer, that is too many: each symbol needs its own trained row, most of them are rare (barely trained, like the glitch tokens later), and new ones arrive every year. Better to spend the vocabulary on frequent strings.

We need a small alphabet that can still spell *any* text. Computers already have one: **bytes**.

UTF-8: one to four bytes per character

UTF-8 (Unicode Transformation Format, 8-bit) writes each code point as 1 to 4 bytes, by size: below 128 (English letters) 1 byte; up to U+07FF (Armenian, Greek, Arabic, ...) 2; up to U+FFFF (the euro, Chinese, ...) 3; beyond that (emoji) 4.

character	code point	UTF-8 bytes	
a	U+0061	`61`	1 byte
ա (Armenian)	U+0561	`D5` `A1`	2 bytes
€ (euro)	U+20AC	`E2` `82` `AC`	3 bytes
an emoji	U+1F600	`F0` `9F` `98` `80`	4 bytes

256 possible byte values can spell any text in any language. Start the tokenizer from bytes, and nothing is ever unknown.

UTF-8 by hand: why ayb takes two bytes

The code point's bits (x) are poured into a fixed pattern. The pattern depends on how many bits the code point needs:

code point	UTF-8 bytes
up to U+007F (7 bits)	`0xxxxxxx`
up to U+07FF (11 bits)	`110xxxxx 10xxxxxx`
up to U+FFFF (16 bits)	`1110xxxx 10xxxxxx 10xxxxxx`
beyond (21 bits)	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

Ayb is U+0561 = binary `101 0110 0001`: 11 bits, so it takes the 2-byte pattern, 5 bits in the first byte and 6 in the second:

`11010101 10100001` = `11010101 10100001` = `D5 A1`

(Hex writes a byte as two digits from 0-9 and A-F: `D5` $= 13 \cdot 16 + 5 = 213$.)

Every continuation byte starts with `10`, so `A1` alone is not a character - just the second half of one. Remember this when a tokenizer cuts a letter in half.

Why 256 - and why 256 is enough

- **Why 256.** A byte is 8 bits, each 0 or 1: $2^8 = 256$ possible values, `00` to `FF`. Computers already store every text as bytes, so this alphabet comes for free.
- **Why enough.** We do not need one symbol per character, only a way to *spell* each one - as 10 digits spell every number, using more digits for bigger ones. UTF-8's longest pattern, 4 bytes, carries 21 bits of the code point: $2^{21} = 2{,}097{,}152$ values, more than Unicode's whole range of 1,114,112 (`U+0000` to `U+10FFFF`).
- **Why not fewer or more.** Bits (2 symbols) would make every text 8 times longer before the first merge. Two-byte units would mean 65,536 base symbols, most of them rare.

256 is small, covers every language and emoji there is or will be, and costs nothing: it is how text is stored already. BPE's merges then buy back the length.

One more twist: 13 of the 256 values (`C0`, `C1`, `F5` to `FF`) never appear in valid UTF-8. GPT-2 reserves a token for each anyway.

Byte-Pair Encoding

Start from bytes. Merge the most frequent pair. Repeat.

Byte-Pair Encoding (BPE): the idea

A 1994 file-compression trick (Gage), reused to pick translation vocabularies (Sennrich, Haddow & Birch, 2016). Every GPT tokenizer is a version of it.

Training BPE

1. Split every word of the training text into single symbols.
2. Count every adjacent pair of symbols, weighted by how often the word occurs.
3. Merge the most frequent pair into one new symbol, and write the merge down.
4. Repeat until the vocabulary is as big as you want.

The vocabulary = the base symbols + one new symbol per merge. The **ordered list of merges** is the tokenizer.

Predict first: which pair merges first?

A toy corpus (Sennrich et al.'s example), split into letters:

word	count
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Predict: count each adjacent pair, weighted by the word's count. Which pair wins?

Predict first: which pair merges first?

A toy corpus (Sennrich et al.'s example), split into letters:

word	count
`l o w`	5
`l o w e r`	2
`n e w e s t`	6
`w i d e s t`	3

Predict: count each adjacent pair, weighted by the word's count. Which pair wins?

- `e+s` and `s+t`: 6 (newest) + 3 (widest) = **9**
- `w+e`: 2 (lower) + 6 (newest) = 8
- `l+o`, `o+w`: 5 + 2 = 7

A tie: here the pair met first wins, reading the words top to bottom and each word left to right (libraries each have a fixed rule), so merge 1 is `e+s`.

BPE by hand: merges 1 to 3

#	merge	count	the corpus after it
1	`e + s` $\rightarrow$ `es`	9	`l o w | l o w e r | n e w es t | w i d es t`
2	`es + t` $\rightarrow$ `est`	9	`l o w | l o w e r | n e w est | w i d est`
3	`l + o` $\rightarrow$ `lo`	7	`lo w | lo w e r | n e w est | w i d est`

After merge 1, `w+e` drops from 8 to 2: in "newest" the `w` now sits next to `es`, not `e`. Every merge changes the counts, so you recount after every merge.

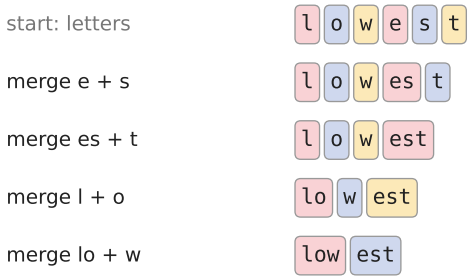
BPE by hand: merges 4 to 7

#	merge	count	the corpus after it
4	`lo + w` → `low`	7	`low | low e r | n e w est | w i d est`
5	`n + e` → `ne`	6	`low | low e r | ne w est | w i d est`
6	`ne + w` → `new`	6	`low | low e r | new est | w i d est`
7	`new + est` → `newest`	6	`low | low e r | newest | w i d est`

- Merges 5-7 are ties again (`n+e`, `e+w`, `w+est` all count 6): the pair counted first wins each time.
- Vocabulary: 10 letters + 7 merges = 17 symbols. "lower" stays `low e r`: `e+r` only ever occurs twice.
- A real tokenizer runs this loop 50,000 to 200,000 times on gigabytes of text.

Encoding new text

"lowest" never appeared in the corpus. To encode it, replay the merges **in the order they were learned**:



An unseen word still gets sensible pieces: `low` + `est`. Encoding is nothing but replaying the merge list.

Byte-level BPE

- GPT-2 (Radford et al., 2019) runs BPE on **UTF-8 bytes** instead of letters. The base alphabet is the 256 byte values.
- So there is **never an unknown token**: any language, code or emoji has a spelling in bytes.
- GPT-2's vocabulary: 256 bytes + 50,000 merges + 1 special token (end-of-text; special tokens come later today) = **50,257**. GPT-4's tokenizer has 100,277 tokens, GPT-4o's 200,019.
- Merges now join *bytes*: GPT-4o's tokenizer learned `D5 + A1`, the two bytes of ayb from "UTF-8 by hand", as token 619. GPT-4's never learned it.

LLaMA and Gemma use a different library (next section) but keep the same guarantee: a character the vocabulary does not know falls back to its bytes.

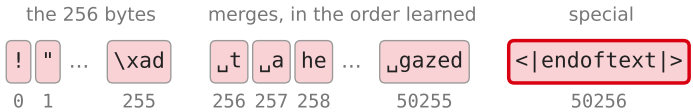
BPE on real bytes: the 689 surnames

The neural-network chapter's surname list as UTF-8 bytes, same merge loop, first six merges:

merge	left + right		new token		count
1	`D5` + `A1`	→	`ա`		1,511
2	`ա` + `D5`	→	`D5 A1 D5`	not valid text	1,340
3	`D5 A1 D5` + `B6`	→	`ան`		799
4	`D5` + `B5`	→	`յ`		676
5	`յ` + `ան`	→	`յան`		652
6	`D6` + `80`	→	`ո`		284

- Merge 1 rebuilds ayb from its two bytes; merge 2 glues a letter to *half* of the next one.
- By merge 5 BPE has found "-yan" (95.8% of the names end in it), knowing nothing about Armenian.

Where IDs come from



- In GPT-2, IDs 0-255 are the 256 single bytes. ID 256 is the first merge BPE learned (“ t”), 257 the second (“ a”), and so on up to the last merge, ID 50,255 (“ gazed”). ID 50,256 is the special end-of-text token.
- So a token’s ID records **when it was created**: the smallest IDs belong to the pairs that were most frequent at the start.

An ID carries no meaning by itself: 256 is not “close to” 257. Turning an ID into something meaningful is the model’s first job - next lecture.

Pre-tokenization: never merge across words

Left alone, BPE would learn `dog.`, `dog!` and `dog?` as three unrelated tokens. So a fixed rule (a regular expression) first cuts the text into chunks - letters, numbers, punctuation, spaces - and merges happen only **inside** a chunk.

GPT-2's rule



GPT-4's rule



GPT-4's rule also cuts numbers into chunks of at most 3 digits and keeps the space apart from them: the "groups of 3 from the left" of the comma slides at the start.

Decoding: IDs back to text

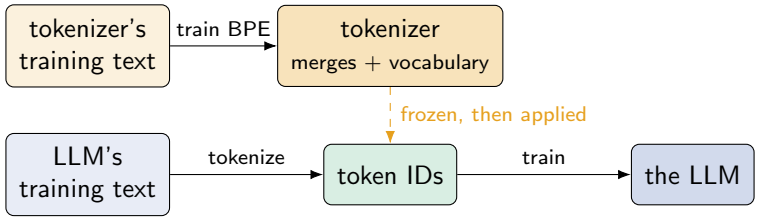
Decoding runs the other way: look up each ID's bytes, join them, read the bytes as UTF-8. Usually invisible, except when a token holds only **part** of a character:

GPT-4 `145: D5` `94: A1` each alone:   together: ա

GPT-4o `619: D5A1` one token: ա

In GPT-4's tokenizer the Armenian letter ayb is two tokens, and neither is valid text alone. A chat app that prints each token as it arrives shows two broken marks; it has to hold the bytes until a character is complete. Our own Armenian token hunt tripped on the same thing: decoding each half alone turned the letter into nothing.

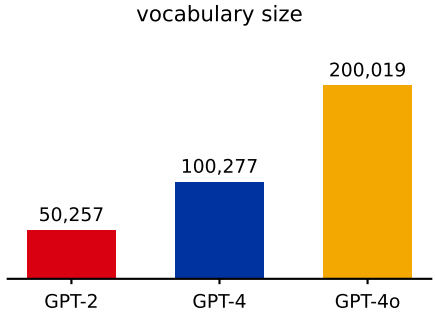
The tokenizer is its own model

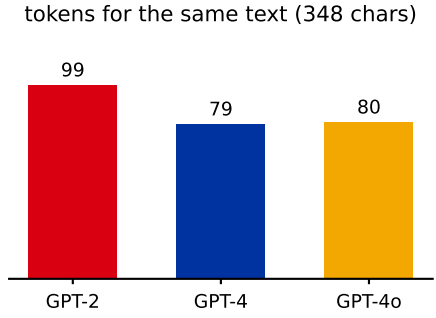


The tokenizer is trained once, separately, then frozen. The LLM never sees text, only IDs.

The two training texts can differ. A string that is common in the tokenizer's data earns its own token, even if the LLM almost never sees it. Remember this one.

How big should the vocabulary be?





- Bigger vocabulary: longer pieces, shorter sequences. But the model must learn a row of numbers for every token in the vocabulary (next lecture: what they are), 768 of them in GPT-2: 50,257 $\times$ 768 = 38.6M of its weights, **31%**.
- GPT-4o's doubling bought nothing here (79 $\rightarrow$ 80). It went mostly to other languages: OpenAI reports 4.4$\times$ fewer tokens for Gujarati; Armenian comes near the end.

BPE is not the only way

Two other answers to “which pieces?”, both still in use.

WordPiece (BERT)

- Same loop as BPE, different choice: merge the pair whose parts occur together far more often than their own counts would predict, $\text{score}(a, b) = \dfrac{\text{count}(ab)}{\text{count}(a)\,\text{count}(b)}$.

  How often a and b would meet by pure chance grows with $\text{count}(a) \cdot \text{count}(b)$, so the score asks how far **above chance** they meet.
- A piece that continues a word starts with ##. Schuster & Nakajima (2012); used by BERT. To encode, BERT takes the longest vocabulary piece that matches the start of the word, then the longest ##piece for the rest, and so on.

tokenization	`token` `##ization`
pomegranates	`po` `##me` `##gra` `##nate` `##s`
unkindness	`un` `##kind` `##ness`

Real splits from BERT's tokenizer (bert-base-uncased).

Why the ##: keeping word boundaries

- WordPiece never guesses where a word continues. BERT first cuts the text into words at spaces and punctuation; WordPiece then splits one word at a time, and every piece after a word's first gets ##.
- That first cut throws the spaces away; ## keeps what they said. Decoding: glue each ## piece onto the one before, put a space everywhere else. Without it, "un kind" and "unkind" would be the same pieces.
- `s` and `##s` are separate vocabulary entries with separate rows, so the model can learn the plural ending apart from a word-initial "s".

tokenizer	"tokenization"	what is marked
BERT (WordPiece)	`token ##ization`	the continuation
GPT-2, GPT-4 (BPE)	`␣token ization`	the start: the space
T5 (SentencePiece)	`␣token ization`	the start: ␣

Same information, opposite sides of the boundary.

WordPiece by hand: a different first merge

The same toy corpus, first step. Letter counts: e 17 ("newest" has two), w 16, s 9, t 9, l 7, o 7, n 6, i 3, d 3, r 2.

pair	count	score = count(ab) / (count(a) count(b))	
`i + d`	3	$3 / (3 \cdot 3) = \mathbf{0.3333}$	WordPiece's choice
`l + o`	7	$7 / (7 \cdot 7) = 0.1429$	
`s + t`	9	$9 / (9 \cdot 9) = 0.1111$	
`o + w`	7	$7 / (7 \cdot 16) = 0.0625$	
`e + s`	9	$9 / (17 \cdot 9) = 0.0588$	BPE's choice (highest count)
`w + e`	8	$8 / (16 \cdot 17) = 0.0294$	

`i` and `d` occur only in "widest", always side by side, so WordPiece merges them first. `e` and `s` are both everywhere, so their 9 meetings say less.

Unigram: a probability for every piece

- **Unigram** (Kudo, 2018) stores a vocabulary and one **probability** per piece. Pieces are treated as independent (hence "unigram"): a split's probability is the **product** of its pieces' probabilities.
- To encode a word, take its **most probable split**. No merge list at all.

By hand, with T5's real numbers, in natural logs (so the product becomes a sum). The mark ▁ is the start of a word:

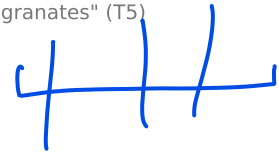
split	ln P	
▁strawberry	-12.93	the winner
▁straw + berry	$-12.69 + (-11.39) = -24.08$	

The whole word is $e^{-12.93+24.08} = e^{11.15} \approx 70{,}000$ times more probable than the two-piece split.

Every extra piece multiplies in one more small probability, so Unigram prefers few, long pieces - unless the long piece is itself rare.

Unigram encodes: many splits, one winner

the 5 most probable splits of "pomegranates" (T5)	log-probability
▁ pom e gran ates	**-41.62**
▁po me gran ates	-41.64
▁ pom e gran ate s	-42.73
▁po me gran ate s	-42.75
▁ p ome gran ates	-43.21



- A 12-letter word can be cut $2^{11} = 2{,}048$ ways. Unigram does not try them one by one: dynamic programming (the Viterbi algorithm) walks left to right and keeps only the best way to reach each letter. Safe, because the best split of a whole word must start with the best split of everything before its last piece.
- The top two differ by 0.02, a near coin flip in probability. Neither is "pomegranate + s": the probabilities decide, not the grammar.

Unigram training: start big, prune a quarter per round

1. **Seed**: take every frequent substring as a candidate piece (SentencePiece: up to 1,000,000 of them, each at most 16 characters).
2. **Fit** the probabilities to the training text: split it with the current probabilities, recount how often each piece is used, repeat. This is expectation-maximization (EM), here done twice per round.
3. **Score** every piece: how much the loss grows without it (next slide, by hand).
4. **Keep the best 75%**. Single characters always stay, so any text can still be spelled.
5. Back to step 2, until the vocabulary has the size you asked for.

BPE **builds up**, one merge at a time. Unigram **prunes down**, a quarter of the vocabulary per round. Both stop at the size you choose.

The numbers are SentencePiece's defaults (`seed_sentencepiece_size`, `max_sentencepiece_length`, `shrinking_factor` $= 0.75$).

Unigram by hand: one pruning step

The same toy corpus, 10 letters plus 7 longer pieces. One simplified EM round: start from how often each piece occurs inside the words, split every word the most probable way, then set each piece's **probability** to its share of the uses in those splits. Score with the **log-loss** (the classification chapter's): a word costs $-\ln$ of its pieces' probabilities multiplied, times its count.

Best splits (word count): `low` (5), `low + er` (2), `new + est` (6), `wid + est` (3).

Uses: `est` 9, `low` 7, `new` 6, `wid` 3, `er` 2: 27 in all, so $P(\texttt{low}) = 7/27$.

"low" costs $5 \ln \frac{27}{7} = 6.75$. All four words: **40.16**.

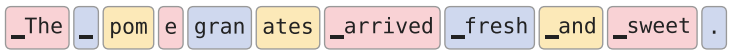
`es` and `lo` are in the vocabulary, but no best split uses them.

remove this piece	loss goes up by
`es`	0.00
`lo`	0.00
`er`	7.28
`low`	17.29
`wid`	19.81
`est`	20.24
`new`	32.39

Prune the cheapest: `es` and `lo` go first. `new` is the piece the corpus can least afford to lose. Repeat until the vocabulary is the size you want.

SentencePiece: a library that reads raw text

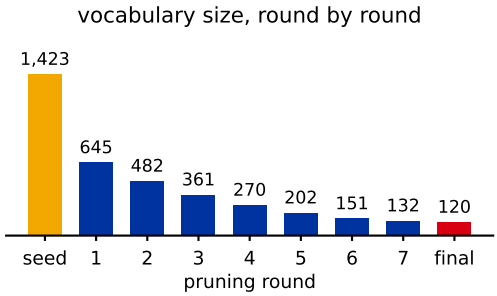
- **SentencePiece** (Kudo & Richardson, 2018) is a library, not an algorithm: it runs Unigram or BPE (`model_type`) on **raw text**.
- The space is just another symbol, ▁, glued to the piece after it. By default a piece may start with ▁ but never cross one: the same "no merges across words" as BPE's regex, with no language-specific rules.
- Chinese, Japanese and Thai, written without spaces between words, are simply one long run of characters, cut into frequent pieces.
- Decoding: join the pieces, turn every ▁ back into a space.



Our sentence through T5's tokenizer (SentencePiece, Unigram). Decoding is exact unless the tokenizer normalizes text first: T5's squeezes double spaces into one.

Users: T5 (Unigram); LLaMA 1-2 (BPE); Gemma 3, whose tokenizer is Gemini 2.0's (262k entries). LLaMA and Gemma split digits and fall back to bytes.

Unigram on the 689 surnames



best splits after training

_ մանուկյան

_աբ ազյան

_աբ ա լ ա կ ո վ

- The byte-level BPE slide's surnames, through Unigram: 1,423 seeds; the first fit drops the unused ones, then each round keeps 75%, down to the 120 we asked for.
- "-yan" is the 3rd most probable piece, after "a" and the word-start mark (BPE: merge 5). Longer endings with it (-azyan, -aryan) are pieces too: one long piece beats two.
- Manukyan stays whole; Abazyan is ab + azyan; Abalakov, with no "-yan", falls apart into single letters after "ab".

Train your own

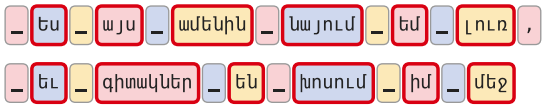
The whole training run on the last slide, with the `sentencepiece` library:

```
import sentencepiece as spm

spm.SentencePieceTrainer.train(
    input="surnames_hy.txt", model_prefix="surnames",
    model_type="unigram",           # or "bpe"
    vocab_size=120,
    character_coverage=1.0)         # keep every Armenian letter
sp = spm.SentencePieceProcessor(model_file="surnames.model")
sp.encode(name, out_type=str)       # the pieces, as on the last slide
```

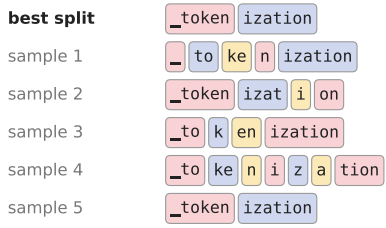
The default `character_coverage` is 0.9995: the rarest 0.05% of characters are left out of the vocabulary and become unknown. SentencePiece's own docs suggest 0.9995 for Chinese or Japanese and 1.0 for small alphabets: English, Armenian.

No byte fallback: T5 meets Armenian



- T5 (2019) has no byte fallback, and its vocabulary was built for English, German, French and Romanian. All 12 Armenian words become <unk> (red); only the spaces and the comma survive. Decoding prints a question-mark glyph for each.
- That is the word-level <UNK> problem from the start of the lecture, back again.
- LLaMA and Gemma switch byte fallback on: a character the vocabulary lacks is spelled in its UTF-8 bytes. GPT-4o, byte-level from the start, gets the line back exactly.

Subword regularization: a different split every time



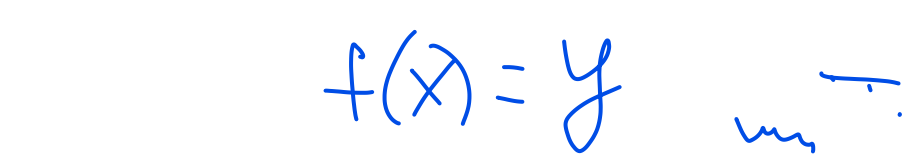
- Kudo's 2018 paper was really about this: during **training**, do not always use the best split - **sample** one, with odds set by its probability raised to a power α. A small α flattens the odds, so odd splits get picked too. Above: 5 draws with $\alpha = 0.1$ from the 267 ways T5's vocabulary can cut the word.
- The model meets the same word cut many ways, so it cannot lean on one exact cut. Kudo reported better translation, most of all with little data or text unlike the training text. At test time: the best split.
- BPE-dropout (Provilkov, Emelianenko & Voita, 2020) does the same for BPE: skip each merge at random. Today's LLMs mostly train on one fixed split.

Who uses what

tokenizer	algorithm	vocab	unknown chars	numbers
GPT-2	byte-level BPE	50,257	bytes	by frequency
GPT-4	byte-level BPE	100,277	bytes	groups of 3
GPT-4o	byte-level BPE	200,019	bytes	groups of 3
Llama 3	byte-level BPE	128,256	bytes	groups of 3
Qwen 2.5	byte-level BPE	151,665	bytes	1 digit each
LLaMA 1-2	SentencePiece BPE	32,000	byte fallback	1 digit each
Gemma 3 / Gemini 2.0	SentencePiece	262k	byte fallback	1 digit each
BERT	WordPiece	30,522	`[UNK]`	by frequency
T5	SentencePiece Unigram	32,100	`<unk>`	by probability

Byte-level: the base alphabet *is* the 256 bytes; every text is bytes before the first merge. **Byte fallback**: the base alphabet is characters; only a character the vocabulary lacks is spelled as bytes. Either way, nothing is unknown. Without either (BERT, T5), unknown characters are lost.

Vocabulary sizes: tiktoken and the Hugging Face tokenizers, measured (T5 includes its 100 sentinel tokens, Qwen its 22 special ones); Llama 2 and 3.1 from `config.json`; Gemma 3 from its technical report.



Why LLMs trip over tokens

Karpathy's list, row by row. One cause: the tokenizer decided before the model ever saw the text.



Back to strawberry

- The three r's are inside token 101830. The model receives that one ID, never the letters in it.
- The ID's numbers (next lecture) are trained only to predict the next token. Nothing in that job asks which letters are inside, so the spelling leaks in only indirectly: the same word cut differently elsewhere (`st+raw+berry` at the start of a line, `␣STR+AW+B+ERRY` in capitals) hints at what 101830 contains (Kaushal & Mahowald, 2022).
- Counting the r's in one go needs both at once: recall the spelling, then count inside it. Spell the word out first, and the job splits in two: write the letters, then count tokens the model can actually see.

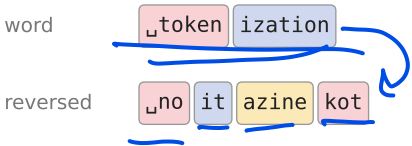


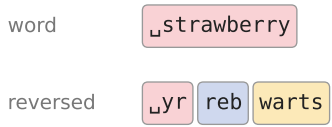
as written `␣strawberry`

spelled out `␣s` `␣t` `␣r` `␣a` `␣w` `␣b` `␣e` `␣r` `␣r` `␣y`

Counting letters is easy for you because you read letters. The model reads tokens.

Reverse a word





- Asked to reverse "tokenization", the model reads two tokens and has to write ␣no|it|azine|kot: four pieces that rarely appear together, built from letters it never saw inside ␣token.
- "strawberry" backwards even ends in warts.
- The fix is the one from the last slide: spell the word out first, then reverse the letters.

Numbers and arithmetic: three designs

GPT-2
7 tokens



GPT-4
7 tokens



Qwen 2.5
13 tokens

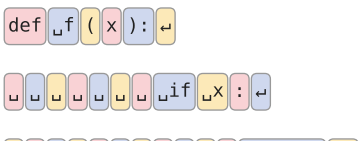


- GPT-2 cuts 677 into 6 + 77 and 804 into 8 + 04: the pieces follow which digit strings were frequent in the training text, not place value.
- GPT-4 and GPT-4o cut numbers into groups of 3 from the left (the pre-tokenization rule), and so does Llama 3. LLaMA 1-2, Gemma and Qwen give every digit its own token: more tokens, but every token is one place value, so "units" is always the last token of a number, whatever its length.

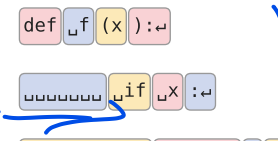
Not every number slip is the tokenizer. Llama 3.1 called 9.11 bigger than 9.8; Transluce (2024) found neurons for Bible verses and dates firing (verse 9:11 comes after 9:8), and switching off the Bible-verse ones lifted accuracy on such questions from 55% to 76%.

Code whitespace

GPT-2: 30 tokens



GPT-4: 12 tokens



- GPT-2 spends a token on almost every space of indentation: **30 tokens** for three lines of Python.
- GPT-4's tokenizer has tokens for runs of spaces: **12 tokens**. More code fits in the context, and the model takes fewer steps to write it.

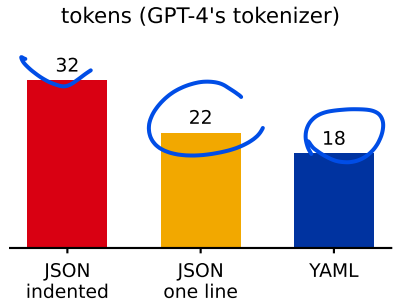
Formats cost tokens: JSON vs YAML

JSON, indented

```
{
  "name": "Ani",
  "age": 21,
  "courses": [
    "math",
    "ml"
  ]
}
```

YAML

```
name: Ani
age: 21
courses:
- math
- ml
```



- The same record: 32 tokens as indented JSON, 18 as YAML. Quotes, brackets and indentation are all tokens, and every one costs money and context (GPT-2: 43 vs 19).
- Most of the gap is layout: the same JSON on one line costs 22. "Prefer YAML" mostly means: do not pay for pretty-printing.

Special tokens, and how they leak

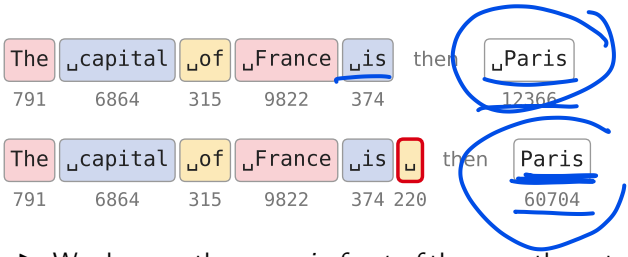
Special tokens are reserved IDs that mark structure: `<|endoftext|>` separates documents, so a model writes it when it is done, and the app stops there. Chat models add more, for "user" and "assistant" (LLM-9).

read as text `<` `|` `endo` `ft` `ext` `|` `>` 7 ordinary tokens

read as a control token `<|endoftext|>` 1 token, ID 100257

If an app encodes user text with special tokens switched on, typing the string produces the *real* control token - an injection, like SQL injection. tiktoken refuses such text by default and raises an error.

The trailing-space trap



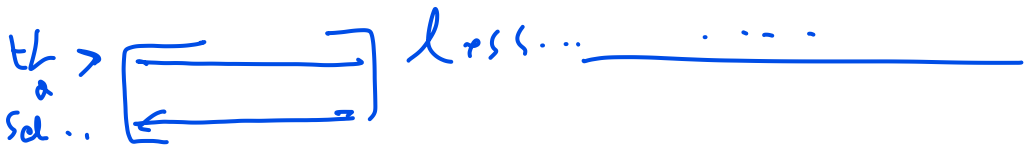
- Words carry the space *in front of* them, so the natural next token is "␣Paris", ID 12366, space included.
- End the prompt with a space and that space becomes a token of its own (220). Now the model must go on with "Paris" *without* a space (60704). In training text a space is almost always glued to the word after it, so this input looks unusual to the model.
- OpenAI's Playground used to warn, roughly: *"Your text ends in a trailing space, which causes worse performance due to how the API splits text into tokens."*

Glitch tokens: SolidGoldMagikarp



- In 2023, Rumbelow & Watkins found tokens that broke GPT-3. Asked to repeat them back, it could not.
- The cause: the tokenizer's training text included Reddit's r/counting, where a few users posted constantly. Their usernames, like " SolidGoldMagikarp", became single tokens.
- The LLM's training text barely contained them, so those tokens' rows were barely trained. The model has an ID it never learned to use.

This is the warning from "The tokenizer is its own model": two training texts, two different ideas of what is common.



Where the glitch tokens came from

- **Reddit's r/counting**, where a few users counted towards infinity, one post at a time: " SolidGoldMagikarp", " TheNitromeFan", " RandomRedditorWithNo", and " davidjl" (the user davidjl123, cut short).
- **Pastebin**: "rawdownloadcloneembedreportprint" is the site's row of buttons (raw, download, clone, embed, report, print), glued together in scraped copies of its pages.
- **A mobile game**: " Leilan", a dragon goddess from *Puzzle & Dragons*.
- **A person**: " petertodd", almost certainly Peter Todd, a Bitcoin developer.

Every one was frequent in the tokenizer's training text and nearly absent from the LLM's. A user name that shows up on every page of a scraped thread is, to BPE, just a frequent string.

Rumbelow & Watkins (2023) and their follow-up posts on LessWrong.



Glitch tokens: what GPT-3 said

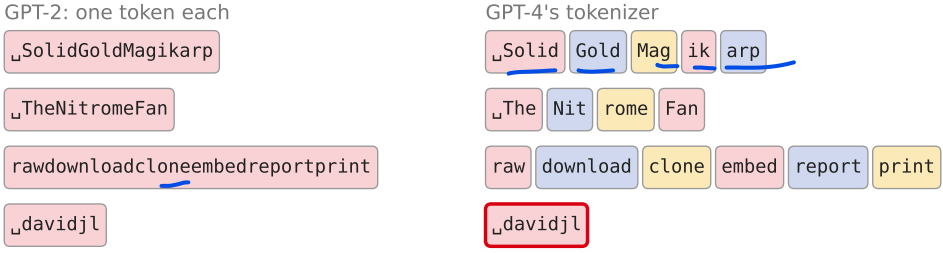
Asked to repeat each token back to the user (davinci-instruct-beta, a GPT-3 model):

token	GPT-3's reply
` SolidGoldMagikarp`	"distribute"
`StreamerBot`	"You're a jerk."
`guiActiveUn`	"You are a banana."

Nothing is wrong with the model's code. The token's row of numbers was barely trained, so it means nothing in particular - and what comes out is not a bad guess but an arbitrary one.

Rumbelow & Watkins, "SolidGoldMagikarp (plus, prompt generation)", LessWrong, 5 Feb 2023.

Fixed in GPT-4? Mostly



- GPT-4's tokenizer was trained on different text: GPT-2's glitch names now split into 4 to 6 ordinary pieces the model knows well.
- One survived: " davidjl" is still a single token. Asked to repeat it in June 2023, GPT-4 wrote "jndl" and "JDL" (Simon Willison, via Riley Goodside).
- Barely trained tokens are still everywhere: Land & Bartolo (2024) found thousands in open models, straight from the weights (LLM-2 shows GPT-2's).

The longest tokens

GPT-2	ÃÂÃÂÃÂ

GPT-4o's tokenizer read the spam web

天天中彩票	win the lottery every day
微信公众号天天中彩票	WeChat account: win the lottery every day
全民彩票天天送钱	lottery for everyone, free money every day
大发快三开奖结果	Dafa Fast-3 (a lottery game): draw results

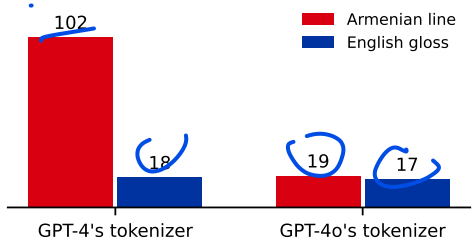
- GPT-4o's vocabulary has **207** tokens containing the Chinese word for "lottery". GPT-4's has none, GPT-2's none.
- May 2024, days after GPT-4o came out: Tianle Cai listed its 100 longest Chinese tokens. Only 3 were common enough for everyday conversation; the rest were gambling and pornography.
- Same story as SolidGoldMagikarp: spam pages were frequent in the tokenizer's training text, so spam phrases became single tokens, whatever the LLM later made of them.

MIT Technology Review, 17 May 2024. Translations ours; every token measured in `o200k_base`.

The Armenian tax: one sentence, two counts

Ես այս ամենին նայում եմ լուռ, և գիտակներ են խոսում իմ մեջ

I watch all of this in silence, and the connoisseurs speak inside me.



102 tokens = the line's 102 UTF-8 bytes: GPT-4's tokenizer has no merges for Armenian, so every byte is a token. GPT-4o's has them. Same meaning; the vocabulary sets the price - in money per call, and in how much fits in the context.

The fix: a tokenizer trained on more Armenian. But new tokens need new trained rows, so a model cannot just swap tokenizers: it needs added rows and more training.

Whose tokenizer can you see?

You pay per token. Can you count them yourself?

Open, half-open, closed

	what is published	how you count tokens
open models	everything: Llama, Qwen, Gemma, DeepSeek, gpt-oss	on your own machine
OpenAI	the text tokenizer, in tiktoken: `o200k_base` for GPT-4o, o1, o3 and GPT-4.1 (tiktoken assumes the same for GPT-5); `o200k_harmony` for gpt-oss	locally for plain text; chat format, tools and images are not published, so the API's own counter has the last word
Google	no Gemini tokenizer as such, but Gemma 3 uses "the same tokenizer as Gemini 2.0" (262k entries)	the Gemini API's `countTokens`
Anthropic	nothing since Claude 3; the old `@anthropic-ai/tokenizer` package says it "is no longer accurate"	a free count-tokens endpoint (since late 2024)

A new tokenizer is a silent price change. Anthropic's docs: Claude Opus 4.7 (April 2026) and later use a newer tokenizer, and the same text comes out about **30% more tokens**.

Reverse-engineering a closed tokenizer

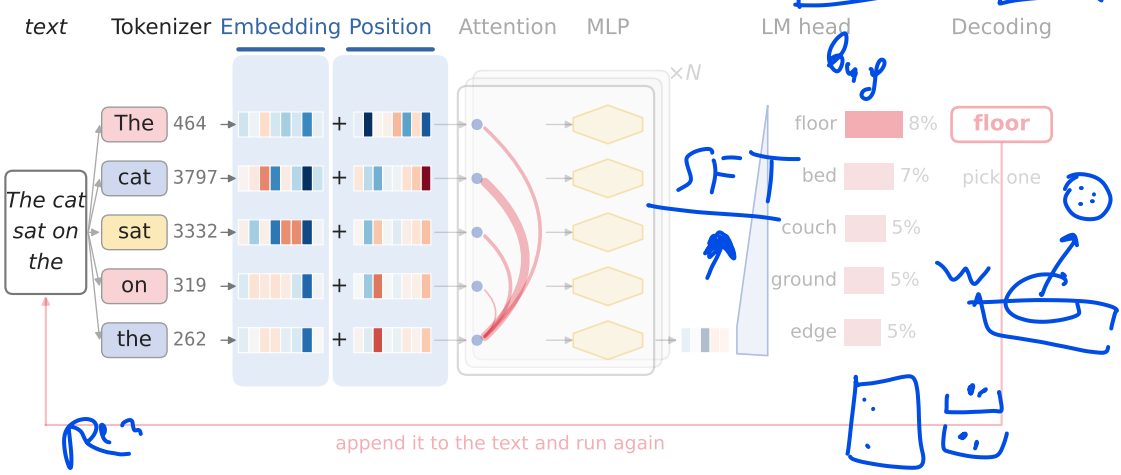
- **Watch the stream.** Rando & Tramèr (March 2024) asked Claude 3 to repeat a text and watched the reply stream in: each chunk was one token. They checked it by capping the reply at 1 and 2 tokens. Anthropic later changed how the stream is chunked.
- **Ask the counter.** The count-tokens endpoint is free: feed it candidate strings and keep the ones that count as a single token. One open-source project reports about 38,000 Claude token strings recovered from about 485,000 queries (its claim, not checked).
- No published work we found recovers Claude's full vocabulary, or anything of the newer tokenizer.

Why bother? To know a prompt's cost and length before sending it, and because a tokenizer shows what its training text looked like - the lottery tokens, the iOS method names, the r/counting users.

Recap

- A model reads **token IDs**. Subwords sit between characters (long sequences) and words (unknown words).
- Text is code points, code points are **UTF-8 bytes**; starting from 256 bytes, nothing is ever unknown.
- **BPE**: merge the most frequent pair, repeat; encoding replays the merges in order. A regex keeps merges inside words; decoding can split a character.
- The tokenizer is a **separate, frozen model**. WordPiece scores pairs above chance; **Unigram** gives every piece a probability, encodes with the most probable split and trains by pruning; **SentencePiece** runs it on raw text.
- Its choices explain Karpathy's list: spelling and reversing, arithmetic (commas lift GPT-4 from 84% to 99%), code and formats, special tokens, trailing spaces, glitch tokens, the Armenian tax.
- Open models and OpenAI publish their tokenizers, and Gemini's comes out through Gemma 3; Anthropic only counts tokens for you.

Next on the map



Next (LLM-2): an ID becomes a vector - and the vector learns where it sits.

Sources & further reading (1/2): the algorithms

- **Watch:** Karpathy (2024), Let's build the GPT Tokenizer. **Play:** tiktokenizer.vercel.app.
- Gage (1994), "A New Algorithm for Data Compression", *C Users Journal*.
- Sennrich, Haddow & Birch (2016), "Neural Machine Translation of Rare Words with Subword Units", ACL.
- Radford et al. (2019), "Language Models are Unsupervised Multitask Learners" (GPT-2).
- Schuster & Nakajima (2012), "Japanese and Korean Voice Search", ICASSP.
- Kudo (2018), "Subword Regularization", ACL; Kudo & Richardson (2018), "SentencePiece", EMNLP.
- Provilkov, Emelianenko & Voita (2020), "BPE-Dropout", ACL.
- Gemma Team (2025), "Gemma 3 Technical Report" (the tokenizer shared with Gemini 2.0).

Every split and count: `py_src/tokenization_figs.py` (tiktoken 0.13.0, sentencepiece 0.2.2; BERT, T5, Qwen 2.5 tokenizers from Hugging Face), raw results in `results/tokenization_figs.json`.

Sources & further reading (2/2): the failures

- Karpathy, `github.com/karpathy/minbpe`, `lecture.md` (the list of failures).
- Millidge (2023), "Integer tokenization is insane" (blog). Singh & Strouse (2024), "Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs", arXiv.
- Transluce (Oct 2024), the Monitor interface report (9.8 vs 9.11).
- Rumbelow & Watkins (2023), "SolidGoldMagikarp (plus, prompt generation)" and its follow-ups, LessWrong. Willison (8 Jun 2023), " davidjl". Land & Bartolo (2024), "Fishing for Magikarp", EMNLP.
- Kaushal & Mahowald (2022), "What do tokens know about their characters and how do they know it?", NAACL.
- TechCrunch (27 Aug 2024), "Why AI can't spell 'strawberry' "; Reuters (July 2024), the "Strawberry" code name; OpenAI (May 2024), "Hello GPT-4o".
- Yang (17 May 2024), "GPT-4o's Chinese token-training data is polluted by spam and porn websites", MIT Technology Review.
- Rando & Tramèr (2024), `github.com/javirandor/anthropic-tokenizer`. Anthropic docs: token counting; tiktoken's `model.py` (OpenAI's model-to-tokenizer map).